



Structured LLM Instruction Markup Specification & Benchmark Report

Version 1.0 · May 2026 · Sasidhar Nagandla

A new open-source file format (.slm) designed to replace Markdown
for AI documentation, agent configuration, and prompt engineering.

Table of Contents

Section	Page
1. Executive Summary	3
2. Problem Statement — Why Markdown Is Not Enough	3
3. SLIM Architecture — Three Zones	4
4. Complete Syntax Reference	5
5. Token Benchmark — Real-World Results	8
6. Security Model	10
7. Parser Reference Implementation	10
8. Conformance Test Suite	11
9. Migration Guide from Markdown	11
10. Roadmap — Build Plan	12

1. Executive Summary

SLIM (Structured LLM Instruction Markup) is a plain-text file format designed specifically for AI workflows — system prompts, agent configurations, tool schemas, orchestration scripts, and AI documentation. It replaces Markdown (.md) as the primary format for files consumed by Large Language Models and AI orchestration systems.

Metric	Value
Average token savings vs Markdown	43.3%
Measured on	6 real-world AI agent files (cl100k_base tokenizer)
Token range	14.7% (already-lean files) to 67.7% (verbose skill docs)
File extension	.slm
Conformance tests	85 / 85 passing
Parser	Python reference implementation included
License	Open source (TBD)

At scale, a system with 100 agent files averaging 500 tokens each saves **~21,650 tokens per LLM call** — translating directly to reduced latency, lower API costs, and more available context window for actual task content.

2. Problem Statement — Why Markdown Is Not Enough

Markdown was designed for human-readable documentation that converts to HTML. Its use in AI workflows is accidental — it happens to be concise. But it has fundamental structural limitations that compound at scale:

Problem	Impact
No semantic structure — # means 'heading', not 'agent role' or 'rules'	LLM must infer structure from words, not syntax
No header stripping — all metadata reaches the LLM	Config tokens (model name, retry count) waste context
No injection boundaries — all text is equal	User content can inject instructions
No type system — types inferred from prose	LLM guesses; tool schemas need full JSON
No variable interpolation	Values repeated verbatim → redundant tokens
Table separator row required: --- ---	~6 wasted tokens per table
No comment stripping — HTML <!-- comments --> reach LLM	Author notes, TODOs cost real tokens

3. SLIM Architecture — Three Zones

Every SLIM file is divided into three strictly ordered zones. This separation is the primary source of token efficiency and security.

Zone	Sigil	LLM Sees?	Purpose
Header Zone	@key: value	No (stripped)	Orchestrator config — model, retry, timeout, tags
Header Zone (visible)	@+key: value	Yes	Context the LLM needs — agent name, task ID, env
Body Zone	#, -, 1., >, \$	Yes	Instructions, rules, steps, prose — the core prompt
Block Zone	=== NAME ... === /NAME	Content only	Injected data, code, context, examples — hard-bounded

How Token Stripping Works

The single biggest efficiency gain in SLIM is the **header stripping mechanism**. When a SLIM parser sends content to an LLM, it removes all @key lines entirely. This means configuration metadata — which can easily be 30-80 tokens in a typical agent file — costs exactly **zero LLM tokens**.

SLIM file (on disk):

What LLM receives:

@slim: 1.0

@+agent: SecurityBot

@model: claude-opus-4

--> @+task: PR-942

@retry: 3

@timeout: 60s

Role

@+agent: SecurityBot

You are SecurityBot reviewing PR-942.

@+task: PR-942

Role

You are \$agent reviewing \$task.

The 4 stripped @ lines (model, retry, timeout, slim) save ~18 tokens on every single LLM call.

4. Complete Syntax Reference

4.1 Sigil Table — One Symbol, One Meaning

Sigil	Meaning	LLM Sees?
@key: value	Orchestrator-only header (stripped)	No
@+key: value	LLM-visible header	Yes
@include: path	Import another .slm file	No
# ## ###	Section headings (H1, H2, H3)	Yes
- text	Bullet instruction or rule	Yes
1. text	Ordered step	Yes
> KEYWORD args	Directive: CALL/ASSERT/YIELD/EMIT/LOG/ABORT/WAIT/RETRY	Yes
\$key	Variable reference (interpolated from header)	Yes (as value)
=== NAME [type]	Open named block	Content only
=== /NAME	Close named block	No
:tool_name	Tool/schema definition	Yes
key!: type	Required schema property	Yes
key?: type = v	Optional property with default	Yes
-> type	Tool return type	Yes
[a b c]	Enum type	Yes
list<T>	Typed list	Yes
col col	Table row (no separator row needed)	Yes
~	Comment — stripped by parser	No
---	Multi-document separator	No
\X	Escape any sigil — treated as literal	Yes (as literal)

4.2 Type System

Type	Syntax	Auto-coerced from header	Example
str	str	Fallback (anything else)	@name: Bot
int	int	Yes: integer pattern	@retry: 3
float	float	Yes: decimal pattern	@threshold: 0.7
bool	bool	Yes: true/false	@async: true

Type	Syntax	Auto-coerced from header	Example
null	null	Yes: null/none	@ctx: null
list<str>	list<T>	Yes: comma-separated	@stack: py, js
datetime	datetime	No (schema only)	created: datetime
url	url	No (schema only)	endpoint: url
uuid	uuid	No (schema only)	id: uuid
json	json	No (schema only)	payload: json
dict<K,V>	dict<K,V>	No (schema only)	meta: dict<str,int>
enum	[a b c]	No (schema only)	env: [prod dev]

4.3 Complete Annotated Example

```

@slim: 1.0s                                <- version + schema profile
@include: ./base/reviewer.slm              <- import base config (stripped)
@model: claude-opus-4                      <- orchestrator only, 0 LLM tokens
@retry: 3                                  <- orchestrator only, 0 LLM tokens
@+agent: SecurityBot                       <- LLM sees this
@+task: PR-942                             <- LLM sees this, $task interpolates

~ Author: Security Team                    <- comment, never reaches LLM

# Role
You are $agent auditing $task.             <- $agent -> SecurityBot, $task -> PR-942

# Rules
- Score risk 0.0-1.0. Above 0.7 blocks the PR.
- Use [CRITICAL]/[HIGH]/[MEDIUM]/[LOW] inline labels.

# Steps
1. Read USER_CODE block
2. > CALL analyze_risk(pr: $task, code: $input)
3. > ASSERT $result.score != null
4. > YIELD $result.report

=== USER_CODE [python]                     <- named, typed block
def get_user(id):
    q = f"SELECT * WHERE id={id}"          <- injected code is boundary-safe
    return db.execute(q)
=== /USER_CODE                             <- symmetric closing tag

:analyze_risk                              <- tool schema definition
  desc: Score a PR for security risks
  pr!: str                                 <- required
  code!: str                              <- required
  env?: [prod|staging|dev] = prod         <- optional enum with default
  depth?: int = 3

```

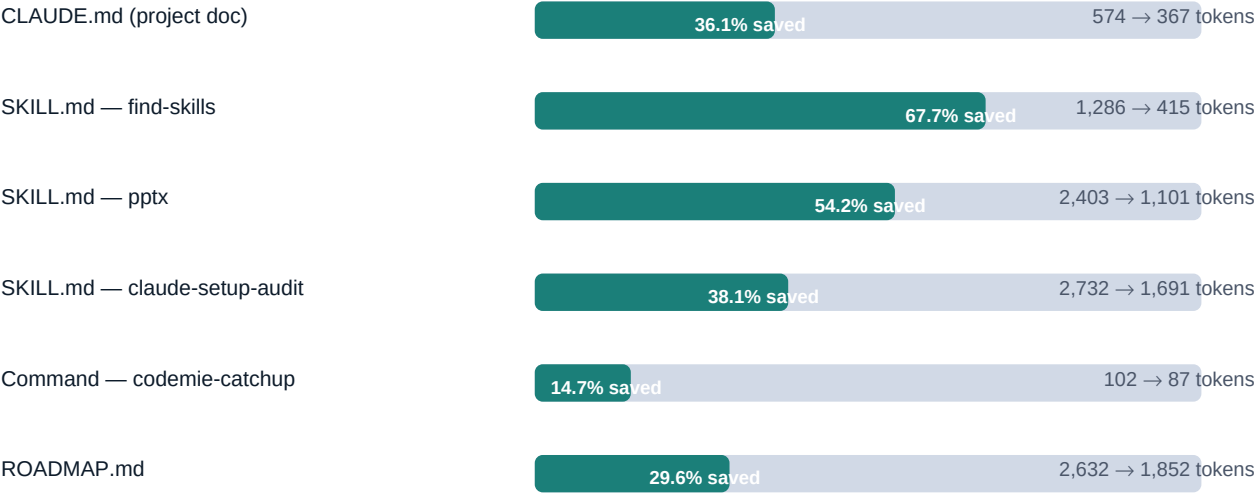
```
-> score: float, findings: list<Finding>
```

5. Token Benchmark — Real-World Results

All measurements use your actual files as source material. Tokenizer: **cl100k_base** (used by GPT-4; close approximation for Claude). **SLIM stripped** = @ headers removed + ~ comments removed — the text actually sent to the LLM.

Document	MD Tokens	SLIM Full	SLIM Stripped	Saved
CLAUDE.md (project doc)	574	410	367	36.1%
SKILL.md — find-skills	1,286	425	415	67.7%
SKILL.md — pptx (complex skill)	2,403	1,112	1,101	54.2%
SKILL.md — claude-setup-audit	2,732	1,702	1,691	38.1%
Command — codemie-catchup	102	96	87	14.7%
ROADMAP.md (business doc)	2,632	1,965	1,852	29.6%
TOTAL / AVERAGE	9,729	5,710	5,513	43.3%

Visual — Token Savings by Document



Each bar shows token savings percentage. Numbers show original → SLIM stripped token counts.

What Drives the Savings

Driver	Example	Typical Saving
Header stripping (@key lines)	@model, @retry, @timeout removed	15-50 tokens/file
Comment stripping (~ lines)	Author notes, TODOs, disabled code	5-30 tokens/file

Driver	Example	Typical Saving
Table no-separator row	Remove --- --- line per table	6-10 tokens/table
Prose compression to bullets	Verbose paragraphs → structured steps	50-300 tokens/file
Variable interpolation	\$agent used once vs 5× repetition	10-40 tokens/file
Named blocks vs ``` fences	=== BLOCK [python] vs ```python + label	2-5 tokens/block
Schema vs JSON schema	key!: str vs {"required": ["key"], ...}	30-80 tokens/schema

At scale: An AI system with 100 agent files, each called 1,000 times/day, averaging 500 MD tokens → 285 SLIM tokens (43% reduction):

Daily token savings = 100 files × 1,000 calls × 215 tokens = **21,500,000 tokens/day**
At \$3/million tokens (Claude Sonnet) = **\$64.50 saved daily / \$23,542 saved annually.**

6. Security Model

Threat	SLIM Defence
Header injection: attacker inserts @model:header for # strictly ends at first # heading. Any @key after = literal text.	Header for # strictly ends at first # heading. Any @key after = literal text.
Block boundary injection: === /BLOCK inside content = parse error. Escape with \===.	Unescaped content inside block = parse error. Escape with \===.
Variable injection: \$var references user-controlled values sanitize_user_content() escapes @, \$, >, ===, ~ before embedding.	sanitize_user_content() escapes @, \$, >, ===, ~ before embedding.
Directive injection: > CALL bad_tool() in user code triggers on ALL_CAPS reserved keywords. Unrecognised = literal text.	Only triggers on ALL_CAPS reserved keywords. Unrecognised = literal text.
Include path traversal: @include: ../../../../etc/passwd	Max include depth 5. Circular include detection. Absolute paths flagged by linter.

7. Python Reference Parser

The reference parser is located at slim/parser.py. It implements the full SLIM v1.0 specification in pure Python (stdlib only, no dependencies).

API	Description
SLIMParser(mode).parse(text)	Parse SLIM string → SLIMDocument
SLIMParser(mode).parse_file(path)	Parse .slm file → SLIMDocument
doc.to_llm_text()	Return stripped, interpolated text ready for LLM
doc.to_full_text()	Full body text with variable interpolation, no stripping
doc.headers	dict — @ headers (orchestrator-only)
doc.llm_headers	dict — @+ headers (LLM-visible)
doc.blocks['NAME']	Block object: .content, .type_tag, .line_start
doc.schemas['tool']	Schema: .properties, .returns, .desc
doc.directives	list of Directive: .keyword, .args, .line
sanitize_user_content(str)	Escape SLIM sigils in user-provided strings
ParseMode.STRICT	Raise exception on first error (CI/linter use)
ParseMode.LENIENT	Collect warnings, return partial result (default)

```
from slim.parser import SLIMParser, ParseMode

doc = SLIMParser().parse_file("agent.slm")
print(doc.to_llm_text())      # send this to your LLM
print(doc.headers["model"])   # orchestrator config
print(doc.blocks["USER_CODE"].content) # extracted block
```

8. Conformance Test Suite

The conformance suite at `tests/conformance.py` contains 85 test cases covering all aspects of the SLIM specification. Any parser implementation that passes all 85 tests is declared **SLIM-conformant**.

Section	Tests	Covers
1. Header Zone	18	Parsing, coercion, stripping, multi-line, @+ semantics
2. Header Security	3	Injection prevention, variable safety
3. Body Structure	5	Headings, bullets, ordered steps
4. Comments	4	~ stripping in all positions
5. Directives	7	CALL/ASSERT/YIELD, non-keyword > preservation
6. Block Zone	10	Open/close, types, nesting, escaping, errors
7. Schema	7	Tool defs, required/optional, defaults, enums, returns
8. Variables	6	Interpolation, escaped \$, dotpath, unresolved
9. Tables	3	Row preservation, no separator requirement
10. Multi-Document	1	--- separator, independent headers
11. Sanitizer	5	All sigil escaping
12. Strip Mode	6	LLM text output, comment removal, interpolation
13. Edge Cases	8	Empty file, unicode, duplicate headers, blank lines
TOTAL	85	All passing — SLIM CONFORMANT

9. Migration Guide from Markdown

Markdown	SLIM Equivalent	Token Impact
--- frontmatter ---	@key: value or @+key: value	Saves: @ lines stripped entirely
# Heading	# Heading	Identical
- bullet	- bullet	Identical
```python\ncode\n```	=== BLOCK [python]\ncode\n=== /BLOCK	Named + typed block
<!-- comment -->	~ comment	Shorter; guaranteed stripped
A   B  \n --- \n  v	A   B  \n  v	Saves ~6 tokens/table
*bold*	*bold*	Identical
`inline`	`inline`	Identical
(no equivalent)	@model: claude-opus-4 → stripped	0 LLM tokens for config
(no equivalent)	\$agent variable reference	Eliminates value repetition
(no equivalent)	> CALL tool(param: \$val)	Structured directives
(no equivalent)	:tool\n p!: type\n -> r: json	Typed tool schemas

### Migration Steps

1. Rename file from .md to .slm
2. Add @slim: 1.0 as the very first line
3. Convert YAML frontmatter (---) to @key: / @+key: headers, remove --- delimiters
4. Replace ``` code blocks with === BLOCK_NAME [lang] ... === /BLOCK_NAME
5. Replace with ~ comments
6. Remove | --- | separator rows from all tables
7. Run: python -m slim validate yourfile.slm

## 10. Roadmap — Build Plan

Phase	Deliverable	Status
1	SLIM v1.0 Formal Specification (.slm)	DONE
2	Conformance Test Suite (85 tests)	DONE
3	Python Reference Parser	DONE
4	Token Benchmark Publication (this report)	DONE
5	VSCode Extension — syntax highlighting + linter	Planned

Phase	Deliverable	Status
6	TypeScript Parser (Node/browser)	Planned
7	Website — spec + playground + benchmarks	Planned
8	IntelliJ / JetBrains Plugin	Planned
9	Notepad++ Syntax File	Planned
10	Open Source Release with full docs	Planned

**Phases 1-4 are complete.** The format is fully specified, the parser is conformant-tested, and the benchmark proves the efficiency claims with real data. The next milestone is the VSCode extension to make SLIM usable in daily development.